

LAB 5 · WEEKS 9–10**Monitoring, Logging & Incident Detection**

Centralised logging, tamper-proof logs, threat detection and incident response — Docker & LocalStack

Lab Learning Outcomes

At the end of this lab, you will be able to:

1. Collect and **centralise logs** from multiple services (cloud telemetry).
2. Distinguish **logs from events** and query logs for security-relevant activity.
3. Build a **tamper-evident (hash-chained)** log and detect alteration.
4. **Detect an incident** by correlating events (e.g. brute-force followed by a suspicious action).
5. Execute the **incident-response** steps: detect, contain, collect evidence, and document a timeline.

Course & Assessment Mapping

Item	Mapping
Course Learning Outcome	CLO2 — Construct secure cloud operations that safeguard data integrity
Lecture topics	Week 6 (Monitoring, Auditing & Management)
Value / skill clusters	VBE3 (Integrity) · SC8 (Integrated Problem-Solving)
Assessment	Lab report + short incident report — contributes to the Lab Assignment

Lab Arrangement (2 Sessions over 2 Weeks)

Session	Week	Focus
Session A	Week 9	Generate and centralise logs; query for failed logins (Tasks 1–3)
Session B	Week 10	Tamper-proof logs, incident detection and response (Tasks 4–6), then the incident report

Note: Session A builds visibility. Session B turns that visibility into detection and response — the ‘prevention eventually fails’ half of security.

Technical Prerequisites

- A laptop with Docker and a terminal.
- **AWS CLI v2** pointed at **LocalStack** (as in Lab 1) — provides CloudWatch Logs.
- Standard shell tools: grep, awk, sha256sum (Git Bash / WSL on Windows).

Security tip: You cannot secure — or prove compliance for — what you cannot see. Logs are foundational to detection, forensics AND compliance evidence (Weeks 6, 10, 11).

Session A (Week 9) — Logging & Centralisation

Setup — Start LocalStack

```
docker run -d --name localstack -p 4566:4566 localstack/localstack
EP='--endpoint-url=http://localhost:4566'
aws $EP logs create-log-group --log-group-name /ccse/app It creates a central place to store application logs.
aws $EP logs create-log-stream --log-group-name /ccse/app --log-stream-name auth
```

Task 1 — Generate Application Logs

Create a **small log of authentication events**, including some failures (an attacker probing).

```
cat > auth.log <<'EOF'
2025-03-01T09:00:01 LOGIN_OK      user=ahmad    ip=10.0.0.5
2025-03-01T09:01:10 LOGIN_FAIL   user=admin   ip=203.0.113.9
2025-03-01T09:01:12 LOGIN_FAIL   user=admin   ip=203.0.113.9
2025-03-01T09:01:15 LOGIN_FAIL   user=admin   ip=203.0.113.9
2025-03-01T09:01:18 LOGIN_FAIL   user=admin   ip=203.0.113.9
2025-03-01T09:01:22 LOGIN_OK      user=admin   ip=203.0.113.9
2025-03-01T09:01:40 EXPORT_DATA user=admin   ip=203.0.113.9 size=500MB
EOF
cat auth.log
```

Task 2 — Centralise Logs (Ship to CloudWatch)

Send each line to the central log service — the cascading-collection idea from Week 6.

```
TS=$(date +%s000) Creates a timestamp in milliseconds. uploading each line from auth.log into CloudWatch Logs / LocalStack Logs.
while IFS= read -r line; do
    aws $EP logs put-log-events --log-group-name /ccse/app --log-stream-name auth \
        --log-events timestamp=$TS,message="$line" >/dev/null; TS=$((TS+1000));
done < auth.log Reads auth.log one line at a time.

# Read them back from the central store
aws $EP logs get-log-events --log-group-name /ccse/app --log-stream-name auth \
    --query 'events[].message' --output text
```

Task 3 — Query for Security-Relevant Activity

```
# How many failed logins, and from which IP?
grep LOGIN_FAIL auth.log | awk '{print $4, $5}' | sort | uniq -c

# Distinguish a log (durable record) from an event (a trigger):
# an EVENT would be 'alert: 4 failures from 203.0.113.9' fired in near real time.
```

Note: End of Session A. Keep auth.log and the centralised read-back. Next week you will make these logs tamper-proof and use them to detect an incident.

Session B (Week 10) — Tamper-Proofing, Detection & Response

Task 4 — Tamper-Proof (Hash-Chained) Logs

An attacker's first move is to edit the logs. Chain each line to the previous hash so any change breaks the chain.

creating a hash chain

```
PREV=0
while IFS= read -r line; do
    PREV=$(printf '%s%s' "$PREV" "$line" | sha256sum | cut -d' ' -f1)
    printf '%s | %s\n' "$line" "$PREV"
done < auth.log > auth.chain
cat auth.chain

# Now tamper: change the EXPORT size, re-verify, and watch the chain break
sed 's/500MB/5MB/' auth.log > auth.tampered
PREV=0; BROKE=no
paste -d'|' <(cut -d'|' -f1 auth.chain) <(cut -d'|' -f2 auth.chain) >/dev/null
```

Recompute the chain from auth.tampered and compare the final hash to auth.chain. A different final hash proves tampering was detected.

Security tip: Store the final hash (or forward the chain) to a separate, append-only location so an attacker who owns the app cannot also rewrite its audit trail (Week 6).

Task 5 — Detect the Incident (Correlation)

No single line was blocked, but together they tell a story. Detect the pattern: repeated failures, then a success, then a large export from the same IP.

incident detection script

```
IP=203.0.113.9
FAILS=$(grep -c "LOGIN_FAIL.*$IP" auth.log) Sets the IP address to investigate.
SUCCESS=$(grep -c "LOGIN_OK.*$IP" auth.log) Counts failed login attempts from that IP.
EXPORT=$(grep -c "EXPORT_DATA.*$IP" auth.log) Counts successful logins from that IP.
echo "IP=$IP fails=$FAILS success=$SUCCESS export=$EXPORT" Counts data export events from that IP.

if [ "$FAILS" -ge 3 ] && [ "$SUCCESS" -ge 1 ] && [ "$EXPORT" -ge 1 ]; then
    echo 'ALERT: probable brute-force -> compromise -> data exfiltration'; Prints the counts.
fi
```

Note: This is what a SIEM does: correlate events across sources into a single detection that no individual log would reveal.

Task 6 — Incident Response

Run the response lifecycle: contain, collect evidence, and document. Work quickly but preserve integrity.

```
# CONTAIN: block the attacker IP (model with an iptables rule)
docker run --rm --cap-add=NET_ADMIN alpine sh -c \ Starts a temporary Alpine container
```

gives it permission to manage network/firewall settings.

Adds a firewall rules

```
'apk add -q iptables; iptables -A INPUT -s 203.0.113.9 -j DROP; iptables -L INPUT -n  
| tail -2'
```

```
# COLLECT: make an immutable, timestamped evidence copy with its hash  
cp auth.log evidence_$(date +%Y%m%d).log  
sha256sum evidence_*.log > evidence.sha256  
cat evidence.sha256
```

Then write a short **incident report** (see deliverables) covering: what happened, how it was detected, what you contained, the evidence collected, and one lesson learned.

Deliverables & Assessment

1. Evidence (label each clearly)

- The centralised get-log-events read-back (Task 2).
- The failed-login count grouped by IP (Task 3).
- The hash-chained log and proof that tampering changes the final hash (Task 4).
- The correlation ALERT output (Task 5).
- The containment rule and the evidence hash file (Task 6).

2. Incident Report (half a page)

Using your findings, write a short report with these headings: **Detection, Analysis, Containment, Evidence & integrity, Lesson learned.**

3. Short-Answer Questions

Q1. What is the difference between a **log** and an **event**? Give an example of each from this lab.

Q2. Why must audit logs be **tamper-proof**, and how does a hash chain achieve this?

Q3. How did **correlation** detect an incident that no single log line revealed?

Q4. List the incident-response steps you performed and the goal of each.

Q5. How do the same logs serve both **security monitoring** and **compliance evidence** (Weeks 6, 11)?

4. Verification Command

```
aws --endpoint-url=http://localhost:4566 logs describe-log-groups  
sha256sum -c evidence.sha256
```

Security Best-Practices Checklist

- ☐ Logs are centralised, not left scattered on each host.
- ☐ Security-relevant activity (failed logins) can be queried.
- ☐ Logs are tamper-evident (hash chain) and forwarded to a separate store.
- ☐ An incident is detected by correlating multiple events.
- ☐ Incident response performed: contain, collect evidence, document.

Cleanup & Teardown

```
rm -f auth.log auth.chain auth.tampered evidence_*.log evidence.sha256  
docker stop localstack && docker rm localstack
```

Expansion Ideas (Advanced Students)

- Stand up a real SIEM stack — **ELK (Elasticsearch, Logstash, Kibana)** or **Wazuh** — in Docker Compose and build a failed-login dashboard.

- Add **Falco** for runtime threat detection and trigger an alert by spawning a shell in a container.
- Automate response: a script that watches the log and blocks an IP after N failures (SOAR-style).
- Configure **log retention** and export to object storage to meet a compliance retention requirement.

References

- Course lecture — Week 6 (Monitoring, Auditing & Management); Weeks 10–11 (compliance evidence).
- Amazon CloudWatch Logs concepts — docs.aws.amazon.com/AmazonCloudWatch/latest/logs
- OWASP Logging Cheat Sheet — cheatsheetseries.owasp.org
- CSA Security Guidance v5 — Security Monitoring domain.